

node2vec y taller de word2vec

Clase práctica 13

Motivación

- ¿Qué eran los **grafos**?
- ¿Cómo podemos introducir información de un grafo **en algoritmos tradicionales** de aprendizaje automático?
- ¿Dónde podemos **seguir aprendiendo** sobre **NLP**?

Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'

Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'
Repaso de grafos	08:05	09:55	15'

Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'
Repaso de grafos	08:05	09:55	15'
<i>Graph embeddings</i>	08:20	10:10	05'

Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'
Repaso de grafos	08:05	09:55	15'
<i>Graph embeddings</i>	08:20	10:10	05'
Breve repaso de word2vec	08:25	10:15	05'

Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'
Repaso de grafos	08:05	09:55	15'
<i>Graph embeddings</i>	08:20	10:10	05'
Breve repaso de word2vec	08:25	10:15	05'
node2vec	08:30	10:20	30'

Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'
Repaso de grafos	08:05	09:55	15'
<i>Graph embeddings</i>	08:20	10:10	05'
Breve repaso de word2vec	08:25	10:15	05'
node2vec	08:30	10:20	30'
Taller de word2vec	09:00	10:50	30'

Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'
Repaso de grafos	08:05	09:55	15'
<i>Graph embeddings</i>	08:20	10:10	05'
Breve repaso de word2vec	08:25	10:15	05'
node2vec	08:30	10:20	30'
Taller de word2vec	09:00	10:50	30'
Más de NLP	09:30	11:20	05'
Cierre	09:35	11:25	05'
Fin de la clase	09:40	11:30	-

Repaso de grafos | Definición

Un **grafo** consiste en una representación de **relaciones** entre **objetos** discretos.

Veamos algunos ejemplos.

Repaso de grafos | Ejemplo #1

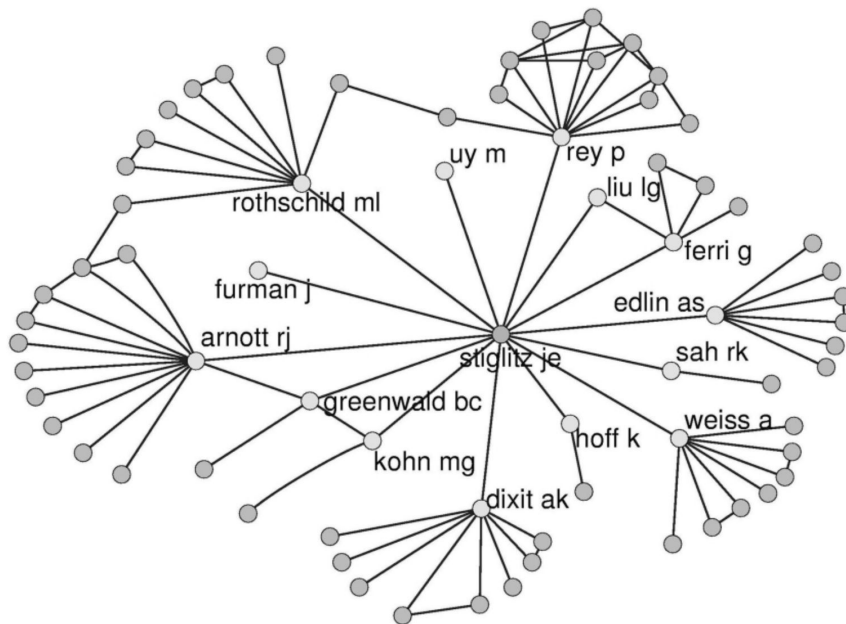


FIG. 2.—Local network of collaboration of Joseph E. Stiglitz in the 1990s. The figure shows all nodes within distance 2 of J. E. Stiglitz as well as the links between them. Some economists might not appear because of misspellings in EconLit. The figure was created by software program Pajek.

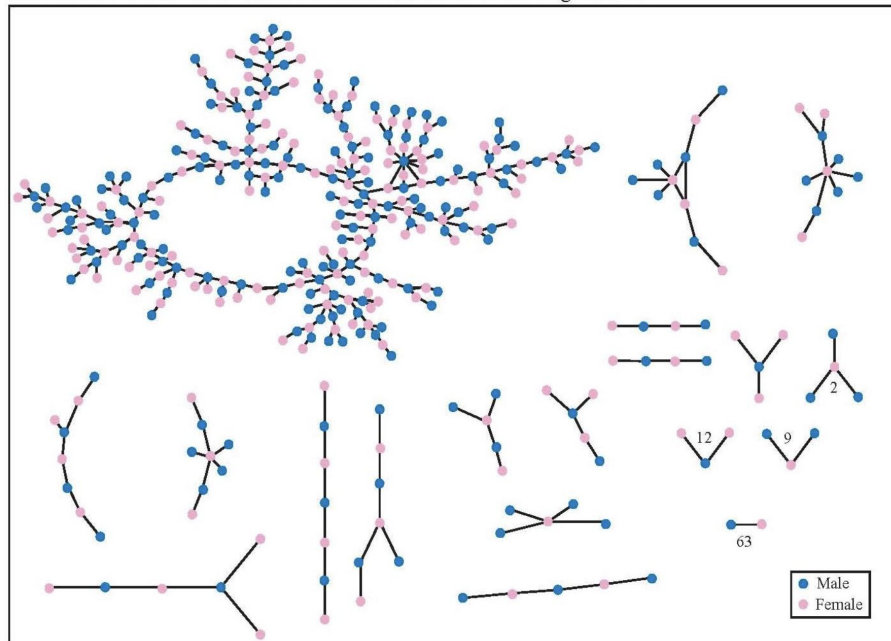
Objetos: **economistas.**

Relaciones: **coautorías.**

[Enlace al artículo académico.](#)

Repaso de grafos | Ejemplo #2

The Structure of Romantic and Sexual Relations at "Jefferson High School"



Each circle represents a student and lines connecting students represent romantic relations occurring within the 6 months preceding the interview. Numbers under the figure count the number of times that pattern was observed (i.e. we found 63 pairs unconnected to anyone else).

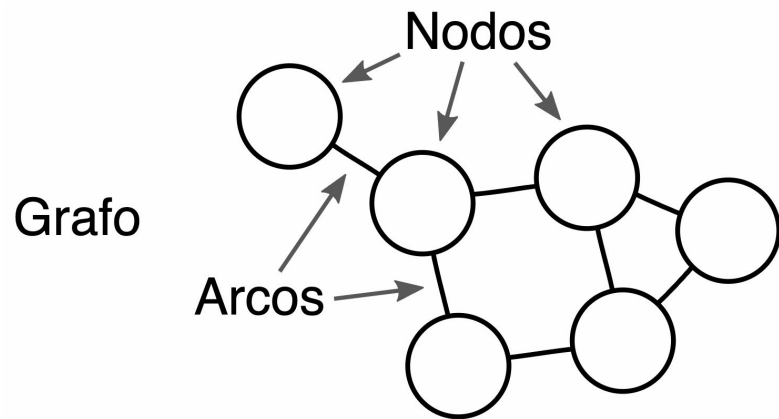
Objetos: **estudiantes de una secundaria.**

Relaciones: **relaciones románticas o sexuales.**

[Enlace al artículo académico.](#)

Repaso de grafos | Terminología

Un **grafo** (G) es un conjunto de **nodos** (V ; la V de vértices; son los objetos) asociado a un conjunto de **enlaces** (E ; son las relaciones; también se les dice arcos, especialmente en matemática) entre los mismos. Entonces: **$G = (V, E)$** .



Repaso de grafos | Terminología (continuación)

Los **nodos** tienen algún **identificador**. Desde un número a algo más específico, como el **nombre del economista** o el **nombre del estudiante**.

Los **enlaces** se definen en función de qué nodos **conectan**.

Tanto los nodos como los enlaces pueden tener asociados **atributos**; es decir, características.

Siguiendo con el ejemplo de los economistas:

- Un posible atributo de un nodo es la **edad del economista**.
- Un posible atributo de un enlace es la **cantidad de artículos escritos juntos**.

Siguiendo con el ejemplo de los estudiantes:

- Un posible atributo de un nodo es la **edad / altura del estudiante**.
- Un posible atributo de un enlace es la **fecha de inicio / hace cuánto tiempo**.

Repaso de grafos | Terminología (continuación)

A su vez, los **enlaces** pueden ser **dirigidos o no**, dando lugar a grafos dirigidos y no dirigidos, respectivamente.

¿Las siguientes relaciones definen grafos dirigidos o no dirigidos?

- Es hermano/a: no dirigido.
- Una dirección URL que lleva de un sitio web a otro: dirigido.

Repaso de grafos | Terminología (continuación)

Otra distinción importante entre grafos es en función de si los **enlaces tienen o no pesos**. En este contexto, se entiende peso como la **fuerza** del enlace.

Recordemos
el grafo de los
economistas.
Es sin pesos.

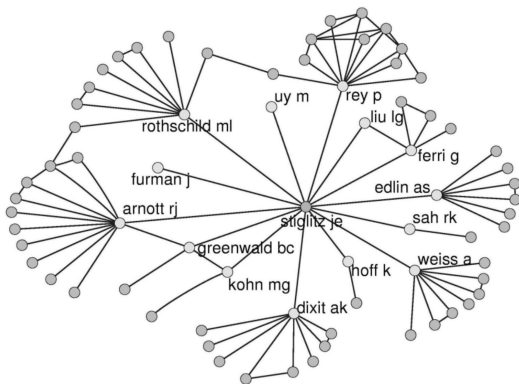


FIG. 2.—Local network of collaboration of Joseph E. Stiglitz in the 1990s. The figure shows all nodes within distance 2 of J. E. Stiglitz as well as the links between them. Some economists might not appear because of misspellings in EconLit. The figure was created by software program Pajek.

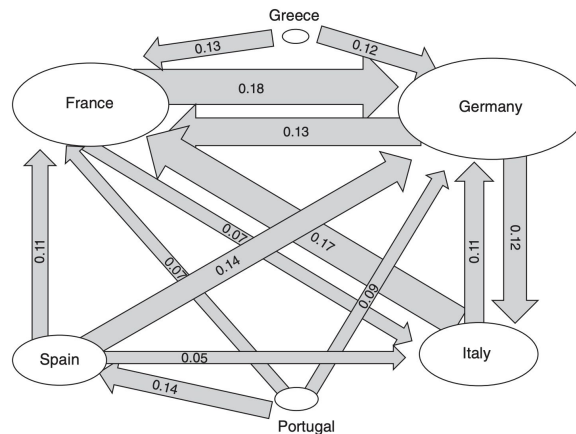


FIGURE 8. INTERDEPENDENCIES IN EUROPE

Notes: The matrix A, describing how much each country ultimately depends on the value of others' debt. The widths of the arrows are proportional to the sizes of the dependencies, with dependencies less than 5 percent excluded; the area of the oval for each country is proportional to its underlying asset values.

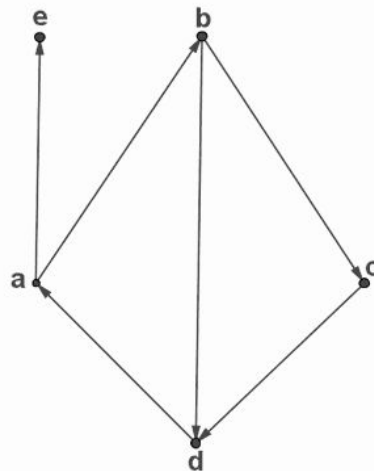
Este otro
grafo es con
pesos.

[Enlace al artículo académico.](#)

Repaso de grafos | Representación

Los grafos pueden ser representados a través de una **matriz de adyacencias** (A), en donde cada **fila i** y cada **columna j** representa un nodo. Los valores de dicha matriz (para una red **sin peso**) se definen así: $A_{ij} = 1$ **si el nodo i tiene un enlace al nodo j** ; 0 en caso contrario.

En cambio, los valores de dicha matriz para una red **con peso** serían los pesos de los respectivos enlaces.



	a	b	c	d	e
a	0	1	0	0	1
b	0	0	1	1	0
c	0	0	0	1	0
d	1	0	0	0	0
e	0	0	0	0	0

Recomendamos ir completando la matriz, a partir del grafo, por fila; es decir, por cada nodo i .

Repaso de grafos | Representación (continuación)

Pero, puede haber **problema**: una matriz de adyacencias para 100 millones de personas tiene 100 millones al cuadrado de elementos. ¡Es **gigante**! Incluso, si la gran mayoría de esos elementos es un 0 (dando lugar a una matriz rala), estaríamos usando una estructura gigante cuando es innecesaria.

Entonces, ¿qué **otra representación** podríamos usar para un grafo?

Una **lista de enlaces**, donde **cada elemento** de una lista sea una **tupla** que indica de qué nodo **surge** el enlace, a cuál se **dirige** y (si es que el grafo es con pesos) el **peso** del enlace.

Graph embeddings

Graph embeddings son las transformaciones **de un grafo a un vector** o conjunto de vectores, capturando información relevante del grafo.

Nosotros vamos a ver un caso particular de graph embeddings: **vertex embeddings**.

En este caso, **cada nodo** pasa a ser **un vector**.

Es mediante estos vectores que podremos introducir en algoritmos tradicionales de **aprendizaje automático** información dada por nodos de un grafo.

Breve repaso de word2vec

Implica entrenar un modelo que busca **predecir** si dos **palabras** tienden a aparecer **juntas**.

Pero, no **interesa** el problema de predicción en sí mismo, sino los **parámetros aprendidos** por este modelo. Estos parámetros estarán contenidos en **dos matrices, T y C**, que contendrán los **word embeddings** (i.e., palabras transformadas en vectores).

El modelo a usar es como una **regresión logística**: un **clasificador** que (idealmente) prediga

- **probabilidades altas** para palabras que **comparten contextos** (casos positivos), mientras que
- probabilidades bajas para palabras que no lo hacen (casos negativos).

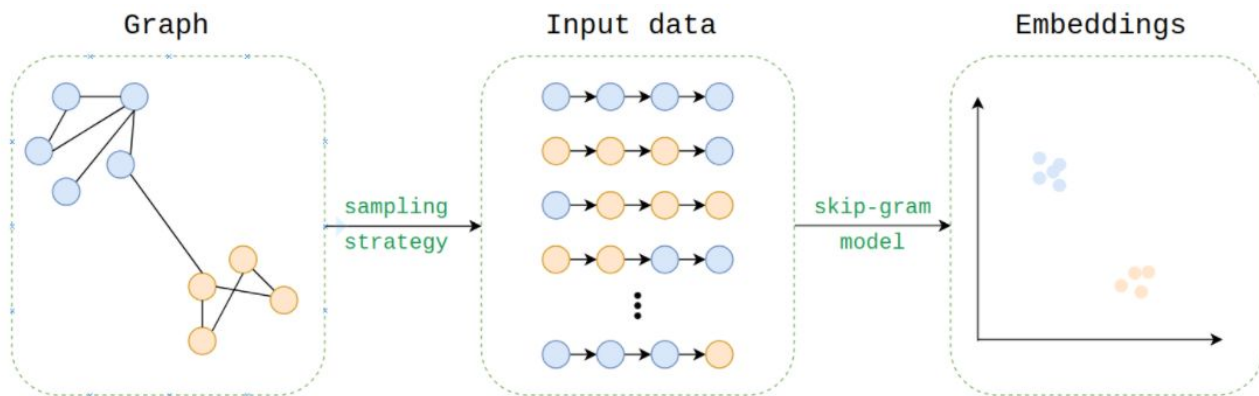
La **hipótesis distribucional** sostiene que **si dos palabras comparten contextos, estas son similares**.

Tomamos dos palabras como similares si el **producto interno** de sus vectores t y c es alto y a este producto, luego, lo transformamos en probabilidad usando la función **sigmoidea**.

Lo que se busca es **encontrar los valores de T y C** que hacen que la probabilidad predicha sea alta para los casos positivos y baja para los negativos.

Al final del proceso, nos quedaremos con los vectores aprendidos.

node2vec | Idea general



A partir de un **grafo**, podemos **simular caminatas** (aleatorias; se las llama *random walks*) a través de los nodos. Se sugiere una **cantidad de caminatas** entre 32 y 64 por nodo y que el **largo de cada caminata** sea de 40 nodos. A cada **nodo** lo podemos pensar como una **palabra** y a cada **caminata**, como una **oración**.

Hecho eso, podemos usar word2vec. El modelo **skip-gram** es un tipo de word2vec que toma una palabra objetivo y busca predecir qué palabras serán sus vecinas.

node2vec | Foco en las caminatas

Dado un **nodo de origen** u , simulamos una **caminata aleatoria de largo fijo** l . c_i denota el nodo i -ésimo en la caminata, empezando con $c_0 = u$. Los nodos c_i son generados siguiendo una distribución específica (fórmula disponible en fuente).

v denota el nodo en que se reside actualmente, mientras que t denota el nodo en el que se residía inmediatamente antes.

Esta caminata aleatoria tiene **2 hiperparámetros**, p y q , que guían el viaje:

- El hiperparámetro de **retorno**, p . Controla la (no-)probabilidad de visitar enseguida un nodo en una caminata.
 - Un **alto valor** ($> \max(q, 1)$) asegura una **menor probabilidad** de muestrear un nodo ya visitado en los siguientes dos pasos (a menos que el próximo nodo en la caminata no tenga ningún otro vecino). Fomenta la **exploración moderada** y **evita la redundancia** en el muestreo.
 - En cambio, un **bajo valor** ($< \min(q, 1)$) conduce a **retroceder** un paso y a mantener la caminata a nivel **local**, cercana al nodo de origen u .
- El hiperparámetro de **entrada-salida**, q . Permite a la búsqueda diferenciar entre nodos de “entrada” y de “salida”.
 - Si $q > 1$, la caminata aleatoria está **sesgada hacia nodos cercanos al nodo t** . De esta manera, las caminatas obtienen una visión **local** del grafo subyacente respecto al nodo de origen en el viaje.
 - En cambio, si $q < 1$, la caminata está más inclinada a visitar nodos que están **más lejos del nodo t** , fomentando la **exploración hacia afuera**.

node2vec | Foco en las caminatas (continuación)

Al ver recién los hiperparámetros, el 1 nunca estaba incluido. ¿Qué pasa entonces cuando **p y q valen 1**? Tiene lugar un caso especial de node2vec conocido como **DeepWalk**, que aprende representaciones de atributos d -dimensionales simulando **caminatas aleatorias uniformes**. Cabe aclarar que la estrategia de exploración de **node2vec tiende a funcionar mejor** que este caso en particular.

El **valor** para asignar tanto a **p** como **q** es **aprendido mediante validación**; por ejemplo, mediante *10-fold CV* realizando una búsqueda en formato grilla sobre $p, q \in \{0,25; 0,50; 1; 2; 4\}$.

En cuanto a la **interacción entre p y q** , tal como vimos, un bajo q fomenta la exploración hacia afuera. Pero, si está acompañado por un bajo p , entonces uno se asegura de que la caminata no se vaya demasiado lejos del nodo de origen.

Por último, las caminatas aleatorias son computacionalmente **eficientes** en términos tanto de **espacio** como de requerimientos **temporales** (más detalles en la fuente).

node2vec | Foco en el modelo skip-gram

El **objetivo** de este modelo está basado en la **hipótesis distribucional**. Recordemos que esta hipótesis establece que palabras en **contextos similares** tienden a tener **significados similares**. Es decir, palabras similares tienden a aparecer en **barrios parecidos** de palabras.

El procedimiento que sigue el **algoritmo** de este modelo es el siguiente. **Escanea** las palabras de un documento y, para cada palabra, busca *to embed it* de forma tal que los atributos de la palabra puedan **predecir las palabras vecinas** (i.e., palabras dentro de alguna ventana de contexto).

node2vec | Modo

Para conocer cómo usar node2vec en Python, veamos `td6-p11-c-node2vec.ipynb`.

Taller | Consignas

Para conocer las consignas, veamos `td6-p11-e-taller-consignas.ipynb`.

Taller | Resoluciones

Para conocer las resoluciones, veamos

`td6-p11-f-taller-resoluciones.ipynb`.

Más de NLP

En caso de que este área de la inteligencia artificial les haya gustado/interesado (o necesiten usarla a futuro para algún trabajo), hay diversos **recursos** para seguir aprendiendo disponibles **en línea ¡y gratuitos!** A continuación, les dejamos algunos:

- El libro completo de Jurafsky y Martin, disponible [acá](#).
- Curso de Stanford “Natural Language Understanding”, disponible [acá](#).
- Curso de Stanford “Natural Language Processing with Deep Learning”, disponible [acá](#).

Cierre

Hoy vimos y practicamos, entre otras cuestiones,

- a modo de repaso, qué eran los **grafos**,
- cómo podemos introducir información de un grafo **en algoritmos tradicionales** de aprendizaje automático y
- dónde podemos **seguir aprendiendo** sobre **NLP**.

Pueden darnos *feedback* de la clase [acá](#).